Context-Aware Travel Itinerary Architect

Multi-Step LLM Agent Report

Raghav Sarna

May 7, 2026

1 Problem statement

Planning a short trip sounds simple until a user specifies concrete preferences (budget, interests, pace, constraints) and the plan must remain consistent with real-world conditions like weather. A single LLM prompt typically produces a plausible itinerary, but it tends to (i) hallucinate current weather or "top places", (ii) miss hard constraints when the prompt becomes long, and (iii) provide an answer that is difficult to programmatically validate or refine.

This project builds a goal-driven travel agent that decomposes itinerary planning into a chained sequence of sub-tasks. Each sub-task is handled by a dedicated LLM call or a real external tool call, and the intermediate artifacts are accumulated in shared state. This design makes the dependency between steps explicit: extraction produces a structured preference schema, tool calls ground the plan in retrieved data, ranking produces a curated candidate set, drafting converts candidates into a day plan, and later steps critique and refine the plan. The result is a structured output (JSON + Markdown) that a user can act on and that can be inspected step-by-step during a live demo.

2 Chain design (what each step does and why it is separate)

The agent is implemented as a modular pipeline in Python with an explicit, traceable data flow through a single `state` dictionary. The main entrypoint (`travel_agent.py`) accepts a natural-language trip description via the terminal and orchestrates the following steps.

Step 1 (LLM: preference extraction). The first LLM call converts raw user text into a strict JSON schema (destination, duration, budget, interests, constraints, travel style, optional start date). This separation matters because later steps require typed fields rather than free-form text; it also enables robust fallbacks when JSON parsing fails.

Step 2 (Tools: weather + places retrieval). External APIs fetch information the LLM should not invent: a short-range forecast (OpenWeatherMap) and candidate attractions (Serper). Downstream steps consume these retrieved artifacts as inputs. Tool failures (missing keys, rate limits) return graceful fallbacks so the chain still runs end-to-end.

Step 3 (LLM: filtering and ranking). Using preferences + retrieved candidates, the LLM categorizes places into `must_visit`, `optional`, and `avoid` with a short reasoning summary. This is intentionally separate from drafting: selection/justification is a different task than sequencing.

Step 4 (LLM: itinerary drafting). The agent converts ranked places into a day-by-day JSON plan (`"Day 1"`, `"Day 2"`, ...) with 4–6 activities per day including meals and transitions.

Step 5 (LLM: weather-aware optimization). The LLM refines timing based only on the retrieved forecast and is explicitly forbidden from distance/geography optimization (a common hallucination mode). It outputs per-day activities plus a weather note.

Step 6 (LLM: critique and refinement). A final reviewer pass checks for overpacked days, missing meals, repetition, and preference mismatches, then returns a refined itinerary along with issues and the changes applied.

3 Tool integration

The pipeline integrates two external tools in Step 2. OpenWeatherMap provides a 5-day/3-hour forecast that is summarized into per-day descriptions and average temperatures. Serper.dev provides a lightweight search API for "top places" queries conditioned on user interests. Tool outputs are inserted verbatim (as JSON) into later prompts, making grounding explicit and demo-time inspection straightforward.

4 Limitations and failure modes

The chain is robust to partial failures, but several limitations remain. LLM JSON is not guaranteed; the implementation uses tolerant parsing plus fallbacks, but fallbacks can cascade into weaker downstream outputs. Tool calls can fail due to missing keys, quotas, or restricted accounts (e.g., Groq `organization_restricted`); the agent logs these and continues. Finally, geography/travel-time optimization is intentionally omitted to avoid hallucinating distances, so the itinerary is plausible rather than route-optimal.

5 Reflection (what I learned and what I would improve)

The main insight is that strict structure at every interface makes LLM pipelines debuggable: fixed schemas reduce ambiguity and contain failures. Tool grounding also changes the task from "generate facts" to "reason over evidence," which is more reliable. Next, I would add lightweight validators (e.g., meals per day, day-count checks) and optional deterministic routing tools while keeping the "no hallucinated distances" constraint, plus persistent logging of intermediate state for demo-time inspection.

References

OpenWeatherMap API documentation: https://openweathermap.org/forecast5.
Serper.dev Search API: https://serper.dev/.
Groq API (OpenAI-compatible endpoint): https://console.groq.com/docs.

A Prompt Design Appendix

B Overview

This appendix supports the Prompt Design deliverable. It lists the full system and user prompts for each LLM step in the chain and explains why the prompts are shaped the way they are, i.e., which constraints they impose and how the next step depends on that structure.

All prompts below are implemented directly in the repository under `steps/`. The agent uses the Groq OpenAI-compatible endpoint for all LLM calls.

C Step 1: Intent & constraint extraction

File: `steps/step1_extract.py`

System prompt (verbatim):

```
You are a travel intent parser. Extract structured travel preferences from user input
    .
Return ONLY a valid JSON object with these exact keys:
{
  "destination": string,
  "duration_days": integer,
  "budget": "low" | "medium" | "high",
  "interests": [list of strings],
  "constraints": [list of strings],
  "travel_style": string,
  "start_date": string or null
}
No explanation. No markdown. Only the JSON object.
```

User prompt (template): `Extract travel preferences from this input: <raw_input>`

Why this shape works: This prompt forces a fixed schema that is easy to validate and consume downstream. Steps 2–6 depend on stable fields like `destination` and `duration_days`. Without a strict schema, later steps would require fragile string parsing.

D Step 3: Filtering & ranking

File: `steps/step3_rank.py`

System prompt (verbatim):

```
You are a travel advisor. Given user preferences and a raw list of places, categorize
    them.
Return ONLY a valid JSON object with this structure:
{
  "must_visit": [list of place names with one-line reason each as "name: reason"],
  "optional": [list of place names],
  "avoid": [list of place names with reason],
  "reasoning_summary": "2-3 sentence explanation of your filtering logic"
}
No markdown. No extra text. Only JSON.
```

User prompt (template): The prompt embeds JSON dumps of: (1) `parsed_preferences` from Step 1, (2) `external_data.places` from Step 2, and (3) `external_data.weather` from Step 2.

Why this shape works: Step 4 should draft from a curated set, not raw search results. Returning categorized lists makes that dependency explicit and prevents Step 4 from "rediscovering" candidates.

E Step 4: Itinerary drafting

File: `steps/step4_itinerary.py`
System prompt (verbatim):

```
You are a travel planner. Create a detailed day-by-day itinerary.
Return ONLY a valid JSON object where keys are "Day 1", "Day 2", etc., each
    containing a list of activity strings.
Include arrival/departure logistics, meals, and transitions. Each day should have 4-6
     activities.
No markdown. No extra text. Only JSON.
Example format:
{
  "Day 1": ["Arrive at airport", "Check in hotel", "Lunch at local restaurant", "
      Visit beach", "Dinner"],
  "Day 2": ["Morning temple visit", "Lunch", "Afternoon market", "Evening show"]
}
```

User prompt (template): The prompt includes destination, number of days, user preferences, and the ranked must/optional/avoid lists from Step 3.

Why this shape works: JSON-by-day is easily post-processed into both a machine-readable artifact (`output.json`) and a human-friendly one (`output.md`). The "4–6 activities + meals" constraint improves completeness.

F Step 5: Weather-aware optimization (no geography hallucination)

File: `steps/step5_optimize.py`
System prompt (verbatim):

```
You are a weather-aware travel optimizer. Your ONLY job is to adjust activity timing
    based on weather.
Rules you MUST follow:
1. Do NOT rearrange activities based on geography or distances -- you don't know
    these.
2. ONLY shift or swap activities based on weather conditions (heat, rain, humidity,
    wind).
3. Prefer outdoor activities during mild/pleasant weather.
4. Move outdoor activities indoors or reschedule if weather is harsh.
5. Add brief weather notes to each day.
Return ONLY a valid JSON object with keys "Day 1", "Day 2", etc.
Each day value is an object: {"activities": [list of strings], "weather_note": string
    }
No markdown. No extra text. Only JSON.
```

User prompt (template): The prompt embeds the draft itinerary (Step 4) and the retrieved weather JSON (Step 2).

Why this shape works: This prompt is intentionally restrictive. In testing, unconstrained "optimization" prompts frequently cause LLMs to invent neighborhood clusters or travel-time reductions. The explicit prohibitions reduce that failure mode and keep the optimization grounded in tool data.

G Step 6: Critique & refinement

File: `steps/step6_critique.py`
System prompt (verbatim):

```
You are a critical travel itinerary reviewer. Evaluate the itinerary and produce a
    refined version.
Return ONLY a valid JSON object with this exact structure:
{
```

```
  "issues": [list of specific problems found],
  "suggestions": [list of concrete improvements],
  "final_changes": [list of changes actually made],
  "final_itinerary": {
    "Day 1": {"activities": [...], "weather_note": "..."},
    "Day 2": {"activities": [...], "weather_note": "..."}
  },
  "summary": "One paragraph summary of the complete trip"
}
No markdown. No extra text. Only JSON.
```

User prompt (template): The prompt includes the optimized itinerary plus the original structured preferences and explicitly asks to check for overpacked days, missing meals, repeated activity types, and mismatches.

Why this shape works: Splitting critique from drafting makes the model behave more like an editor. The explicit `final_changes` field creates an audit trail that is helpful for the live demo.

H Prompt iteration example (change after testing)

Early testing showed that a weaker Step 5 optimizer prompt often performed "geographic" clustering (e.g., grouping places by areas it assumed were nearby) or invented travel-time savings, which the agent cannot validate. The Step 5 system prompt was tightened into an explicit rule list that forbids distance-based rearrangement and restricts changes to weather-based timing. This iteration reduced hallucinated geography and made the chain more defensible: optimization is grounded in retrieved forecast data rather than in unstated assumptions.